

Wstęp teoretyczny i założenia do projektu

1. Charakterystyka problemu

Hasła w rzeczywistych systemach nie są przechowywane w postaci jawnej, lecz jako wynik działania funkcji skrótu. System nie zna bezpośrednio tekstu hasła, a jedynie jego hash. Projektowany program nie będzie odwracał funkcji haszującej, lecz będzie generował kolejne hasła, obliczał ich skróty i porównywał je z hashem docelowym. Problem można skutecznie rozproszyć na wiele maszyn. Klienci mogą niezależnie sprawdzać możliwe hasła, a serwer jedynie zarządza przydziałem pracy i zbiera wyniki.

2. Podstawy teoretyczne

2.1. Funkcje skrótu

Funkcja skrótu przekształca dane wejściowe dowolnej długości do ciągu o ustalonej długości. W projekcie można wykorzystać przykładowo SHA-256, ponieważ jest łatwa do użycia w testach laboratoryjnych.

2.2. Atak słownikowy

Metoda słownikowa polega na sprawdzaniu haseł znajdujących się w przygotowanym zbiorze często spotykanych słów i ich wariantów. Do słownika można dodawać modyfikacje, na przykład cyfry na końcu, wielkie litery lub znaki specjalne. Zaletą tej metody jest szybkie uzyskiwanie wyników dla słabych haseł. W systemie rozproszonym słownik można dzielić na fragmenty i przydzielać różnym klientom.

2.3. Atak brute force

Metoda brute force polega na generowaniu wszystkich możliwych kombinacji znaków z ustalonego alfabetu, na przykład: małe litery, wielkie litery, cyfry oraz wybrane znaki specjalne. Jest to metoda pewna, ale kosztowna obliczeniowo. Aby podzielić pracę między klientów, wszystkie możliwe hasła można ponumerować. Serwer przydziela wtedy każdemu klientowi zakres numerów do sprawdzenia. Dzięki temu każdy komputer sprawdza inną część możliwych haseł i praca jest równo podzielona.

2.4. Algorytmy planowane do wykorzystania

Jednym z nich jest funkcja skrótu SHA-256, która będzie używana do obliczania hashy generowanych kandydatów na hasło. Obliczony hash będzie następnie porównywany z hashem docelowym, aby sprawdzić, czy znaleziono właściwe hasło.

Podstawową metodą wyszukiwania będzie brute force, czyli generowanie wszystkich możliwych kombinacji znaków z ustalonego alfabetu i sprawdzanie. Metoda ta pozwala sprawdzić wszystkie możliwe hasła.

Drugą metodą będzie atak słownikowy, polegający na sprawdzaniu haseł znajdujących się w przygotowanym pliku ze słownikiem. Metoda ta umożliwi szybkie znalezienie prostych i często używanych haseł.

Do rozdzielania pracy między komputery zostanie zastosowany podział zakresowy. Serwer będzie przydzielał klientom zakresy kombinacji do sprawdzenia, dzięki czemu praca zostanie rozdzielona pomiędzy wiele maszyn.

Dodatkowo planowane jest wykorzystanie wielowątkowości po stronie klienta, co pozwoli na równoległe wykonywanie obliczeń i lepsze wykorzystanie mocy obliczeniowej procesora.

3. Dotychczasowe podejścia i inspiracje

W praktyce łamanie haseł bywa realizowane z użyciem narzędzi takich jak Hashcat czy John the Ripper. Celem projektu nie jest odtworzenie pełnej funkcjonalności profesjonalnych narzędzi, lecz zbudowanie uproszczonego systemu.

Z punktu widzenia przedmiotu najistotniejsze jest pokazanie mechanizmów systemu rozproszonego: komunikacji klient-serwer, synchronizacji pracy, przydzielania zadań, logowania wyników oraz pomiaru czasu wykonania.

4. Proponowana architektura rozwiązania

Najbardziej praktycznym wariantem jest architektura klient-serwer. Jeden komputer pełni rolę serwera, a pozostałe komputery klienta. Serwer utrzymuje listę połączonych hostów, rozdziela zadania i odbiera wyniki częściowe.

Klienci są odpowiedzialni wyłącznie za wykonywanie obliczeń. Po uruchomieniu łączą się z adresem IP i portem serwera, zgłaszają gotowość do pracy, pobierają paczkę danych, wykonują obliczenia i zwracają informację o zakończeniu zakresu lub znalezieniu poprawnego hasła.

Pliki konfiguracyjne będą wykorzystywane do przechowywania parametrów działania systemu. W pliku znajdują się: adres IP serwera, numer portu, alfabet znaków, minimalna i maksymalna długość hasła, tryb pracy (brute force lub słownikowy) oraz hash hasła, które ma zostać odnalezione.

4.1. Komunikacja i komunikaty

W systemie przewidziane są następujące komunikaty:

TASK - wysyłany przez serwer do klienta. Zawiera informacje o trybie pracy (np. brute force lub słownikowy), początkowym indeksie, liczbie kombinacji do sprawdzenia oraz parametrach zadania. Komunikat ten przydziela klientowi fragment pracy.

RESULT - wysyłany przez klienta po zakończeniu zadania. Zawiera status wykonania, czas obliczeń oraz ewentualnie znalezione hasło.

STOP - wysyłany przez serwer w celu poinformowania klientów o zakończeniu obliczeń, na przykład po odnalezieniu hasła.

5. Pomiary wydajności

Projekt powinien umożliwiać sprawdzenie, jak liczba komputerów i wielkość paczek zadań wpływają na czas obliczeń. Dlatego należy zapisywać: całkowity czas zadania, czas obliczeń po stronie klienta, liczbę sprawdzonych kandydatów oraz nazwę komputera wykonującego dane zadanie. Ponieważ zegary na różnych komputerach mogą się różnić, najbezpieczniej mierzyć czas całkowity po stronie serwera oraz czas obliczeń lokalnych na kliencie.

6. Zakres funkcjonalny projektu

- Aplikacja serwera uruchamiana z konsoli, z możliwością ustawienia portu i parametrów zadania.
- Aplikacja klienta uruchamiana z konsoli, z możliwością podania IP serwera i portu.
- Obsługa trybu słownikowego i brute force.
- Dynamiczny przydział paczek zadań klientom, którzy zgłoszą gotowość.
- Logowanie wyników do pliku: host, czas, zakres, status, ewentualnie znalezione hasło.
- Możliwość uruchamiania na różnej liczbie komputerów.

7. Przypadki użycia

- Użytkownik uruchamia serwer i ustawia parametry zadania.
- Użytkownik uruchamia klientów na kolejnych komputerach i wskazuje adres IP serwera.
- Klient dołącza do systemu i pobiera paczkę obliczeń.
- Klient kończy zakres i pobiera następny fragment pracy.
- Klient znajduje poprawne hasło i raportuje wynik do serwera.
- Serwer zatrzymuje pozostałych klientów i zapisuje końcowy raport z testu.

Doprecyzowanie założeń, szkielet aplikacji

8. Cel projektu

Celem projektu nie jest faktyczne złamanie hasła, lecz analiza wydajności systemu rozproszonego.

System ma umożliwiać pomiar:

- liczby sprawdzonych haseł na sekundę
- wpływu liczby komputerów na wydajność
- wpływu granulacji (wielkości paczek zadań)
- oszacowanie narzutu komunikacji i synchronizacji

Testy będą wykonywane przez określony czas (np. 2-3 minuty), a następnie analizowane będą wyniki.

9. Podział pracy w zespole

R:

- implementacja serwera
- zarządzanie zadaniami
- obsługa komunikacji

K:

- implementacja klienta
- logika brute force i słownika
- wielowątkowość

D:

- pomiary wydajności
- logowanie wyników
- analiza danych i przygotowanie raportów

10. Szczegóły funkcji skrótu SHA-256

SHA-256 jest kryptograficzną funkcją skrótu. Generuje hash o długości 256 bitów (32 bajty), niezależnie od długości danych wejściowych.

Właściwości:

- deterministyczność - te same dane wejściowe zawsze dają ten sam hash
- jednokierunkowość - nie da się odtworzyć danych na podstawie hashu
- odporność na kolizje - bardzo trudno znaleźć dwa różne wejścia o tym samym hashu
- efekt lawinowy - mała zmiana danych wejściowych całkowicie zmienia hash

Zastosowanie w projekcie:

- generujemy kandydat hasła
- obliczamy jego SHA-256
- porównujemy z haszem docelowym
- jeśli się zgadzają - hasło znalezione

Największym kosztem obliczeniowym jest hashowanie, dlatego opłaca się porównywać jeden hash z wieloma kandydatami.

11. Granulacja zadań

Granulacja oznacza wielkość paczki pracy przydzielanej klientowi. Mała granulacja (np. kilkanaście haseł) pozwala lepiej rozdzielać pracę między komputery i szybciej reagować na zakończenie obliczeń, ale powoduje częstą komunikację z serwerem. Duża granulacja (np. tysiące haseł) ogranicza komunikację i pozwala dłużej liczyć bez przerw, jednak w przypadku wolnego lub uszkodzonego klienta część pracy może być długo blokowana. Celem jest znalezienie optymalnej granulacji jako kompromisu między czasem komunikacji a czasem obliczeń.

12. Sposób wyznaczania zakresów brute force

W celu podziału pracy przestrzeń przeszukiwania traktowana jest jako uporządkowany zbiór wszystkich możliwych kombinacji dla danego alfabetu i ustalonej długości hasła.

Dla każdej długości hasła kombinacje numerowane są niezależnie, traktując je jako liczby w systemie pozycyjnym o podstawie równej liczbie znaków alfabetu.

Serwer przydziela klientom zakresy indeksów dla konkretnej długości hasła. Klient na tej podstawie wyznacza hasło początkowe i generuje kolejne kombinacje.

Zakresy te dzielone są na jednostki pracy o stałej wielkości (granulacji), co pozwala na równomierne rozłożenie obciążenia między klientów.

13. Rozszerzony atak słownikowy

Atak słownikowy nie ogranicza się do prostych słów. W projekcie przewiduje się:

- dodawanie cyfr: hasło1, hasło123
- zmiana wielkości liter: Hasło, HASŁO
- dodawanie znaków specjalnych: hasło!, hasło@
- zamiana znaków: a@, o0, s\$

Z jednego słowa można wygenerować tysiące wariantów, ale w projekcie wystarczą podstawowe modyfikacje.

14. Synchronizacja słownika

W systemie przewidziano mechanizm synchronizacji słownika:

- serwer przechowuje aktualną wersję słownika
- każda wersja posiada timestamp
- klient przy uruchomieniu pobiera słownik lub jego aktualizację
- serwer wysyła klientowi fragment słownika w postaci zakresu

15. Obsługa błędów i timeoutów

Potencjalne problemy:

- klient się zawiesił
- klient się rozłączył
- zadanie trwa zbyt długo

Zostanie wprowadzony timeout. Jeśli klient nie odpowiada zadanie o danym identyfikatorze zostanie uznane za nieukończone i trafi do ponownego przydziału.

Dzięki wykorzystaniu identyfikatora zadania możliwe jest również wykrywanie duplikatów wyników. Jeśli serwer otrzyma odpowiedź dla zadania, które zostało już wcześniej zakończone, wynik taki może zostać zignorowany.

16. Strategia pomiarów wydajności

System będzie uruchomiony na określony czas (np. 2-3 minuty), klienci wykonują obliczenia i raportują wyniki:

- liczba sprawdzonych haseł
- liczba przetworzonych paczek
- czas całkowity po stronie serwera
- czas obliczeń po stronie klienta
- nazwa hosta
- liczba klientów

Na tej podstawie można obliczyć:

- przepustowość - liczba sprawdzonych haseł na sekundę
- przyspieszenie - stosunek czasu wykonania na jednym komputerze do czasu wykonania na wielu komputerach
- efektywność - stosunek przyspieszenia do liczby komputerów, pokazujący straty wynikające z komunikacji i synchronizacji

Nazwa hosta pozwala na analizę wydajności poszczególnych komputerów i identyfikację wolniejszych maszyn.

17. Automatyzacja testów

Klienci po uruchomieniu na wszystkich komputerach automatycznie łączą się z serwerem i oczekują na zadania. Rozpoczęcie testu odbywa się po stronie serwera, który inicjuje przydział pracy. Dzięki temu nie ma potrzeby ręcznego synchronizowania startu aplikacji na wszystkich komputerach.

Parametry połączenia (adres serwera i port) są przechowywane w pliku konfiguracyjnym, co umożliwia ich łatwą zmianę bez modyfikacji kodu.

Wyniki są zapisywane po stronie serwera, co eliminuje konieczność zbierania danych z poszczególnych komputerów i umożliwia wygodną analizę.

18. Szkielet aplikacji

Zaimplementowano wstępny szkielet systemu w architekturze klient-serwer z wykorzystaniem .NET 10, języka C# oraz komunikacji gRPC. Serwer został zrealizowany jako aplikacja ASP.NET Core gRPC Service, natomiast klient jako aplikacja konsolowa.

Serwer:

- uruchamiany jako aplikacja gRPC
- nasłuchuje na określonym porcie
- przyjmuje połączenia klientów
- wysyła komunikaty
- odbiera odpowiedzi

Klient:

- aplikacja konsolowa
- łączy się z serwerem
- odbiera komunikaty
- odsyła odpowiedzi